

SchedulingBuddy - A Discord Bot Utilizing a Natural Language Processor to process a Large Language Model

Brandon Morrow

CSI-4130 - Artificial Intelligence / Tianle Ma / 1 December, 2025

1. Introduction

People increasingly rely on digital calendars, memory, or physical notes to manage tasks and appointments. However, manually creating calendar events, opening an app, selecting dates, setting times, and typing descriptions can disrupt progress on tasks at hand, interrupting workflow. With advancements in large language models (LLMs), applications can now understand natural-language instructions and automate routine tasks.

The objective of this project is to build *Scheduling Buddy*, a Discord bot that listens for trigger words (meet, schedule, etc.), interprets the user's text using an LLM, extracts the date, time, and description, and automatically generates a corresponding event in Google Calendar. The system aims to support human-like phrasing such as:

- "Remind me to call Sarah next Friday afternoon."
- "Dentist appointment January 3rd at 8 AM."
- "Schedule a meeting three hours from now."

The major challenges addressed in this project include:

- (1) Interpreting ambiguous temporal expressions
- (2) Generating correctly structured calendar data
- (3) Ensuring time-zone consistency
- (4) Integrating external services (Discord and Google Calendar).

This project demonstrates how LLM-based natural language interfaces can improve usability and accessibility in productivity software.

2. Related Work

Several commercial systems perform natural-language scheduling including Siri (Apple), Google Assistant, and Cortana (Microsoft). These systems typically rely on proprietary NLP pipelines or pre-trained language models fine-tuned specifically for date-time extraction (e.g., Duckling, Chrono, or in-house solutions).

Academic work in temporal expression recognition (TIMEX) has produced rule-based and machine-learning models for parsing time expressions. Tools like **SpaCy** are commonly used in information extraction tasks. LLMs however have more recently shown state-of-the-art capability in interpreting context-dependent temporal references without explicit rules.

Unlike existing commercial assistants, *Scheduling Buddy* uses a general-purpose LLM NLP model with carefully engineered prompting, demonstrating that even without fine-tuning, modern models are capable of reliably converting informal text into structured scheduling data. This project fits within ongoing research exploring LLM use in real-world productivity tools and autonomous task assistants.

3. Data

This project does not use a traditional dataset. Instead, the “data” consists of real-time natural-language prompts written by the user in Discord. Example prompts include:

- “Schedule call with recruiter on Monday at 10.”
- “Workout session at 5 PM this Saturday.”
- “Add a reminder for mom’s birthday on March 2nd.”

These prompts were tested across a wide range of variations:

- absolute dates,
- relative dates (“in two days”),
- weekday references,
- explicit times,
- ambiguous times (“in the morning”),
- multi-part descriptions.

No labeled dataset was required because the model was used in zero-shot mode, not strictly interpreting a previously existing data set.

4. Methods

4.1 System Architecture

The system consists of four components:

1. Discord Bot

- Listens for the command
`!schedule <message>`

OR

- Listens for trigger words with date and time:

“Lunch tomorrow at 12pm”

- Sends the message to the LLM.

2. LLM Parsing Module

- Uses GPT-based structured output (JSON schema).

Extracts:

- `event_title`
- `event_date`
- `event_time`
- `event_description`

3. Time Normalization Layer

- Converts model output into:

- standardized ISO-8601 timestamps
- Eastern Time (EST) as the reference zone
- Uses `dateutil` and Python's `datetime` libraries.

4. Google Calendar Integration

- Uses OAuth credentials from Google Cloud Console.
- Submits properly formatted event data to the Calendar API.

4.2 LLM Prompting Approach

The LLM was prompted using a strict JSON schema to ensure structured output:

```
{
  "event_title": "string",
  "date": "YYYY-MM-DD",
  "time": "HH:MM",
  "description": "string"
}
```

This prevents ambiguous responses and makes parsing deterministic.

The prompt includes:

- examples of good outputs
- instructions for interpreting relative dates
- instructions for handling missing times (default: no time event)
- instructions to localize to US/Eastern time.

4.3 Calendar Event Construction

Events are constructed as:

```
{
  "summary": title,
  "description": description,
  "start": {
    "dateTime": "...",
    "timeZone":
"America/New_York"
  },
  "end": {
    "dateTime": "...",
    "timeZone":
"America/New_York"
  }
}
```

End times are inferred by adding one hour unless the user specifies a duration.

4.4 Implementation Details

Languages & Frameworks

- Python 3.10
- Discord.py
- Google API Python Client
- dateutil
- OpenAI Python SDK

Deployment Environment

- Local development machine
 - GitHub repository for version control
 - `.env` file for secrets management
-

5. Experiments & Results

5.1 Evaluation Strategy

Because this system generates events rather than numerical outputs, evaluation is primarily qualitative. The model was tested on 50+ natural-language scheduling commands covering:

- simple events,
- multi-word descriptions,
- ambiguous temporal cues,
- relative time expressions,
- events spanning different months,
- commands referencing “today,” “tomorrow,” or “next week.”

5.2 Accuracy Findings

Interpreting Dates:

The model correctly understood natural-language dates in 46/50 tested cases (92%). Errors usually occurred with ambiguous phrases like “this Friday” late at night when the date had already rolled over.

Interpreting Times:

Time extraction accuracy was ~90%. The

remaining failures involved vague expressions like “in the evening,” which required more explicit prompting.

Time Zone Handling:

After improving the pipeline, EST detection worked consistently. The system correctly created events aligned with the user’s actual current date and time.

5.3 Qualitative Examples

User Input	Model Output (Date/Time)
“Plan vet appointment next Monday at 9 AM.”	2025-02-10 09:00
“Dinner with Mike tomorrow night.”	7 PM default time
“Schedule gym session in 3 hours.”	Current time + 3 hours
“Mom’s birthday March 2.”	All-day event on March 2

5.4 Failure Cases

- Ambiguous expressions like “sometime this afternoon” produce inconsistent times.
 - Event duration is sometimes misinterpreted.
 - Occasionally creates two events if the LLM responds with duplicate timestamps in early testing.
 - Casual phrasing (“later tonightish”) can confuse the model.
-

6. Conclusion

Scheduling Buddy successfully demonstrates how LLMs can interpret natural-language scheduling commands and automate calendar creation. The final system integrates Discord, an LLM, time-normalization logic, and the Google Calendar API into a cohesive tool. Experiments show strong qualitative performance, with high accuracy for date and time extraction and reliable creation of calendar events.

Limitations:

- Occasional ambiguity in vague time expressions
- No explicit handling of multi-day events
- Reliance on zero-shot model prompting instead of fine-tuned data
- Needs better error recovery and validation

Future Work:

- Fine-tune a model specifically for temporal extraction tasks

- Add support for reminders, recurring events, and durations
 - Extend beyond Google Calendar (Outlook, Apple Calendar)
 - Add a web dashboard for event review and editing
 - Improve LLM safeguards and timestamp verification
-

References

1. Google Developers. *Google Calendar API Documentation*.
2. OpenAI. *OpenAI API Reference*.
3. Chin, J., et al. "TIMEX Extraction Approaches in NLP." *ACL*, 2020.
4. Facebook Duckling. *Temporal Expression Parsing*.
5. SpaCy Documentation. *Rule-based Matching and NLP Pipelines*.